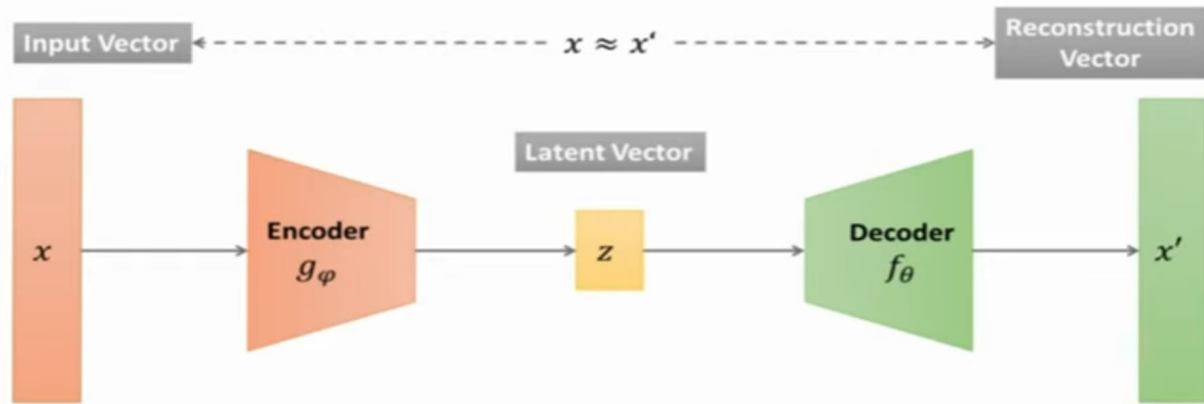
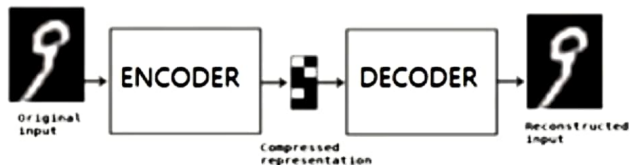

Autoencoders

Architecture



- An auto encoder is a simple (Relatively simple) ML algorithm which acquires input image and it will reconstruct the same. I.e. the image is compressed.
- This is also called as dimensionality reduction.
- Well, The **dimensionality reduction** certainly is used in the data preprocessing (we reduce/compress)
- The process of dimensionality reduction reduces the dimensionality of the considered dataset.
- There could be many features in the considered dataset. Do we use all those features? Not exactly. We use some features of them and such features are to be identified.
- Dimensionality reduction certainly helps in identifying the features which are useful (i.e. of interest).
- Auto encoders are meant for this Dimensionality Reduction.





Contd.,

- Don't we have techniques to do this already?
- We do have!
- We know what PCA – **Principal Component Analysis (PCA)**. They do dimensionality reduction.
- The conversion from higher dimension to lower dimension happens. They also with PCA try to preserve the features which are important. This done with **Linear Transformation**. But, **Auto encoders are using Non – Linear transformation**.
- That's the major difference.

How autoencoders work?



- we know, Auto encoders are feedforward networks
- As discussed, Auto encoders has a relatively simpler architecture and layers. Actually speaking, it has three layers. But, we never count the input layer into account, we call auto encoders as two layered.
- Also, remember this point!
- How many layers are there in Auto Encoders?
 - Let us make it 3 (Actually, as per guidelines it is 2, to avoid confusions, we make it 3)
 - **Input Layer**
 - Hidden Layer (People refer it as bottleneck as well)
 - Output Layer

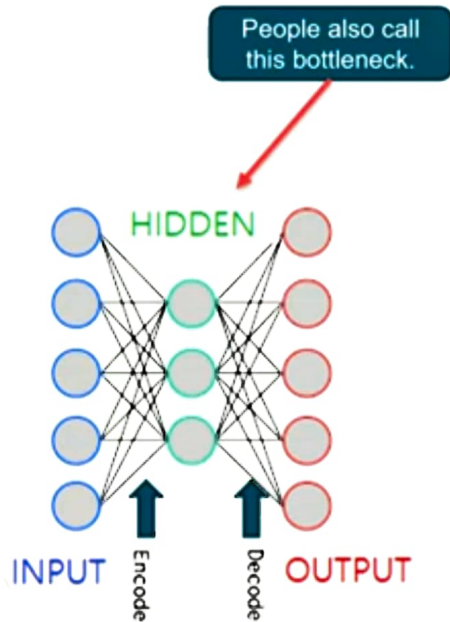


Contd.,

- In Auto encoders, remember this!
- The output should always be the same as input.
- For an instance if the input is set as 1 1 1 1 0 0 0 0 0 the output should (the networks shall try) be 1 1 1 1 0 0 0 0 . (We shall understand this a little better, later)
- Why is this imposed? Simple. The number of neurons should remain consistent.
- Well, Can take an example?

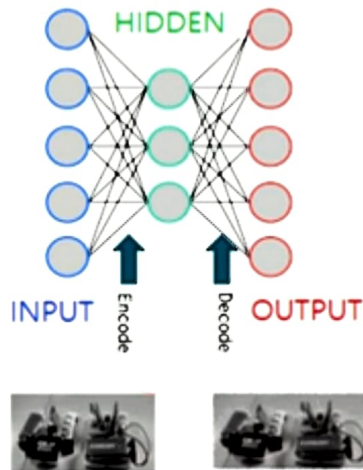
Contd.,

- We can understand a little more now..
- Refer to the simple Auto Encoder Architecture presented in the RHS (Don't worry about the name and other details of the same)
- It has **two layers** (three, input ignored). It has one hidden layer as you could see.
- An instance for a quicker reference. Assume we feed an image with **8 pixel values as input** to the Auto Encoder.
- This is now **compressed** by the **encoder**, to **5 Pixels (Let's say) at the hidden layer i.e. bottleneck layer or middle layer**.
- **Now, comes the decoder. It will try to reconstruct the 5 pixels to 8. (See, it will try to!!)**



The Features / Characteristics of Auto Encoders

- **Data-specific:** This can work only on the data which are similar to what the **system is already trained on**. (Also, this is not like GZIP or WINRAR, where the compression happens and packaging is done) For an instance, if an autoencoder is trained on compressing cat images may not work fine with donkey images.
- **Lossy:** Well, the expectation may not always happen. Same is the case with Auto encoders. The output may not be exact as input. But, it will be a very closer ones. It will certainly be a degraded version. No doubt. This is **lossy** version. And if you want lossless compression, well, find out different methods.
- **Unsupervised / Self supervised** – We can call this unsupervised as we need not do anything other than feeding the raw input. No explicit labeling required. To be precise – They are Self Supervised. They generate own labels from training.



A better pic

